Intelligent Vision Systems

CS773-2026-T9

Forward & Backward Projection & Tsai Calibration Stereo Calibration Errors & Distortion & Distortion Removal

Tony Wang, Shahrokh Heidari, Patrice Delmas

Overview

- Coordinate System Conventions in Camera Geometry
- Backward Projection (example)
- Tsai Calibration Error
    - In 2D image
    - In 3D cube
    - Stereo Calibration Error
- Distortion Removal (examples)

Notation Summary

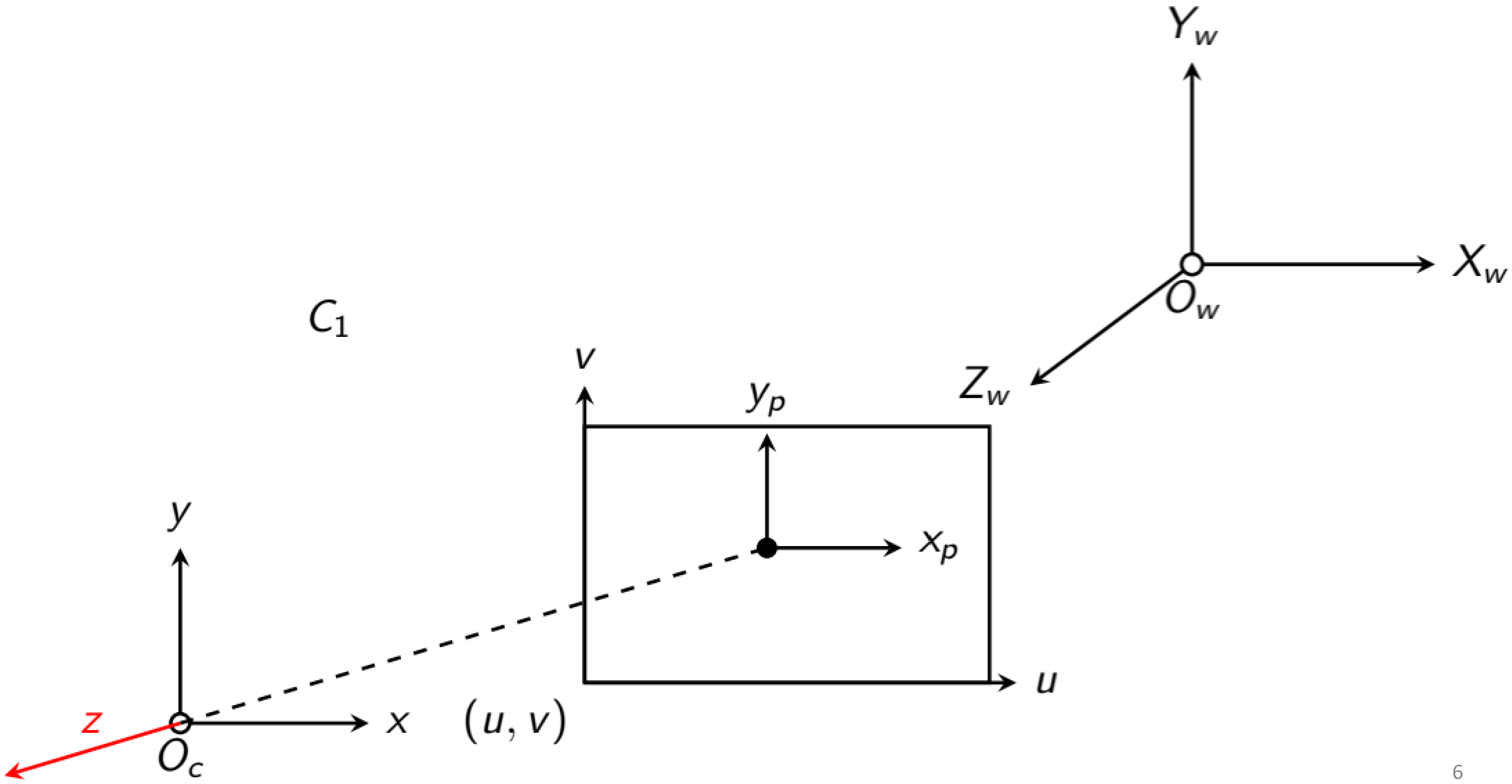
Symbol	Coordinate Space	Meaning
(X, Y, Z)	World Reference Frame	3D world coordinates
(x, y, z)	Camera Reference Frame	3D camera coordinates
(x_p, y_p)	Image Plane	Coordinates in mm
(u, v)	Image Coordinates	Pixel coordinates
f	Camera Model	Focal length (mm)
f_x, f_y	Intrinsic Matrix	Focal length (pixels)
(c_x, c_y)	Intrinsic Matrix	Principal point
R, t	Extrinsic Parameters	Rotation and translation
K	Intrinsic Matrix	Camera internal parameters
P	Projection Matrix	Full camera matrix
s	scalar	Homogeneous scale factor

Notation Summary

Notation	Description
$M_w = (X, Y, Z)^T$	3D calibration cube point in the World Reference Frame (WRF)
$\widetilde{M}_w = (\widetilde{X}, \widetilde{Y}, \widetilde{Z})^T$	Estimated or back-projected 3D point in the WRF
$(u, v)^T$	Measured image point coordinates in pixels
$(\widetilde{u}, \widetilde{v})^T$	Projected image point coordinates estimated from calibration parameters
$(x_p, y_p)^T$	Image coordinates in metric units (mm) relative to the image centre
$(X_c, Y_c, Z_c)^T \sim (x, y, z)^T$	3D point coordinates in the Camera Reference Frame (CRF)
O_c	Optical centre of the camera
f	Camera focal length (mm)
d_x, d_y	Horizontal and vertical pixel size (mm/pixel)
(c_x, c_y)	Principal point/image centre in pixels
$R = [r_{ij}]$	3×3 rotation matrix between WRF and CRF
$t = (t_x, t_y, t_z)^T$	Translation vector between WRF and CRF
K	Intrinsic calibration matrix
$P = K[R\|t]$	Perspective projection matrix
κ_1	First-order radial distortion coefficient
e_i	Calibration error associated with point i
WRF	World Reference Frame
CRF	Camera Reference Frame

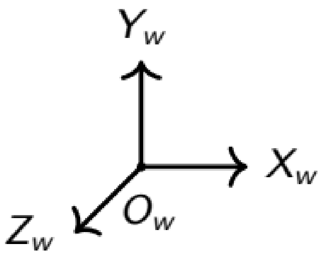
Coordinate System Conventions in Camera Geometry

Coordinate System Conventions in Camera Geometry – C_1



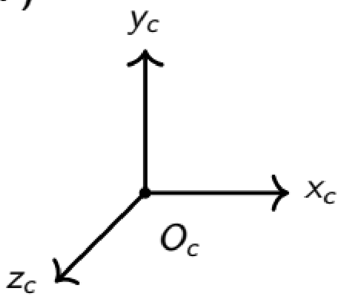
Coordinate System Conventions in Camera Geometry – C_1

World Reference Frame (WRF)



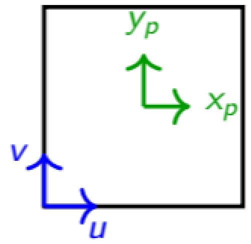
- Global 3D world reference frame (WRF) centered at O_w.
- Used to describe scene geometry.
- Camera pose expressed relative to WRF.

Camera Reference Frame (CRF)



- Camera Reference Frame (CRF) centered at O_c.
- Origin located at optical center.
- Related to WRF.

Image Coordinate Systems



- Green: image-centered coordinate system (x_p, y_p) with origin at the image center (principal point).
- Blue: pixel coordinate system (u, v) with origin at the bottom left corner of the image.

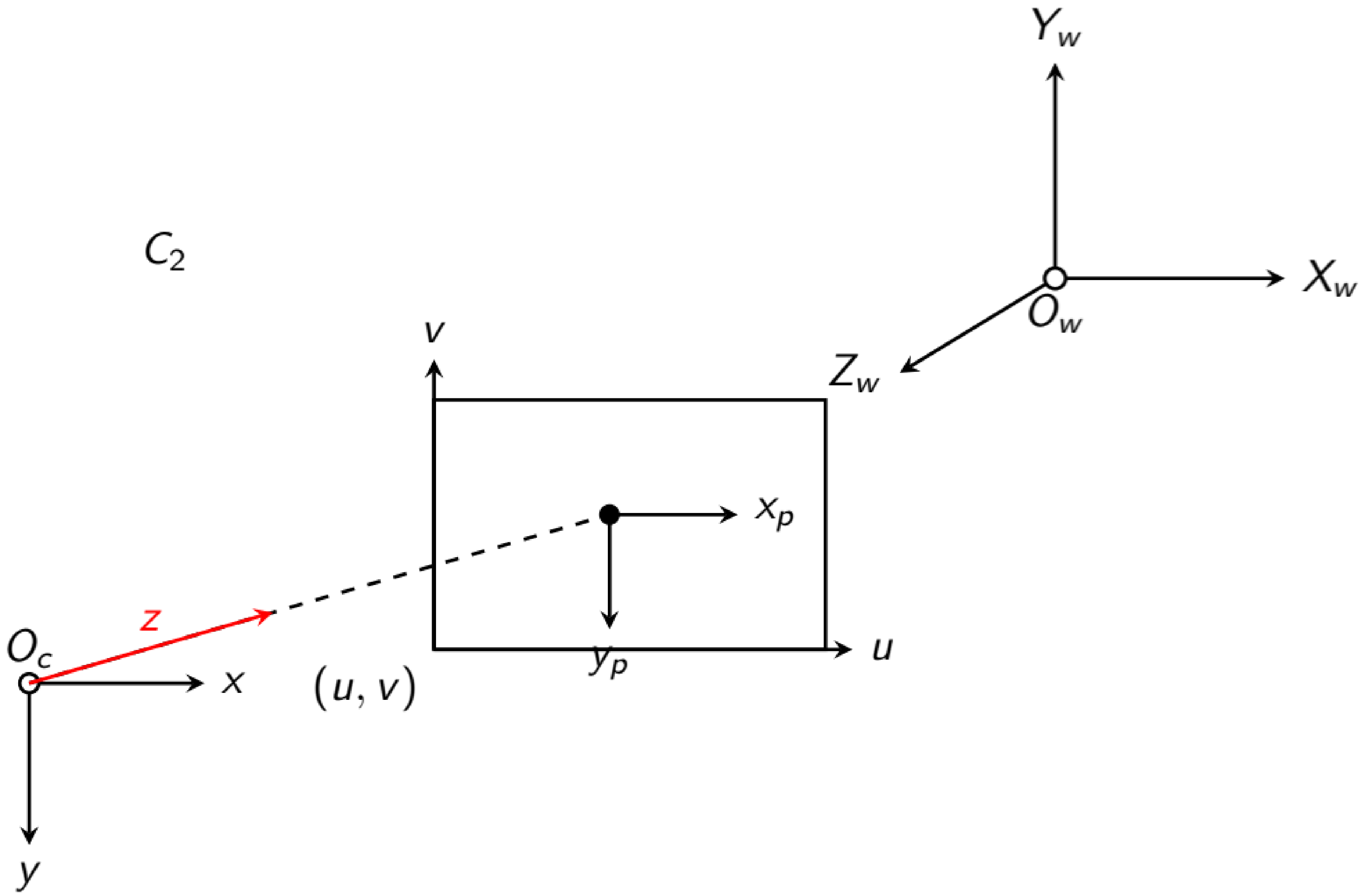
$$\begin{pmatrix} s\tilde{u} \\ s\tilde{v} \\ s \end{pmatrix}_{3\times1} = \overbrace{\underbrace{\begin{pmatrix} \frac{s_x}{d_x} & 0 & c_x \\ 0 & \boxed{\frac{1}{d_y}} & c_y \\ 0 & 0 & 1 \end{pmatrix}_{3\times3} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}_{3\times4}}_{\text{Camera Matrix } (K)} \underbrace{\begin{pmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}_{4\times4}}_{\text{Extrinsic parameters/matrix } (M)}}^{\text{Projection Matrix}} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}_{4\times1}$$

Positive sign

When changing the coordinate from pixel units to mm units:

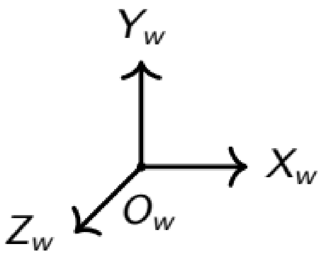
$$x_p = s_x d_x (u - c_x)$$
$$y_p = d_y (v - c_y)$$

Coordinate System Conventions in Camera Geometry – C_2



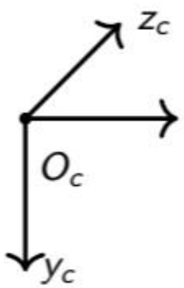
Coordinate System Conventions in Camera Geometry – C_2

World Reference Frame (WRF)



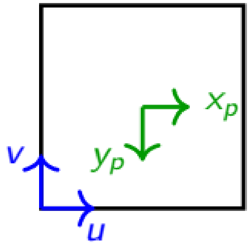
- Global 3D world reference frame (WRF) centered at O_w.
- Used to describe scene geometry.
- Camera pose expressed relative to WRF.

Camera Reference Frame (CRF)



- Camera-centered 3D coordinate system
- Origin located at optical center
- Related to WRF by:

Image Coordinate Systems



- Green: centered image coordinates
- Blue: pixel coordinates
- (u, v) origin at image corner
- (x_p, y_p) centered on principal point

$$\begin{pmatrix} s\tilde{u} \\ s\tilde{v} \\ s \end{pmatrix}_{3\times1} = \overbrace{\underbrace{\begin{pmatrix} \frac{s_x}{d_x} & 0 & c_x \\ 0 & \boxed{-\frac{1}{d_y}} & c_y \\ 0 & 0 & 1 \end{pmatrix}_{3\times3} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}_{3\times4}}_{\text{Camera Matrix } (K)} \underbrace{\begin{pmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}_{4\times4}}_{\text{Extrinsic parameters/matrix } (M)}}^{\text{Projection Matrix}} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}_{4\times1}$$

Negative sign

When changing the coordinate from pixel units to mm units:

$$x_p = s_x d_x (u - c_x)$$
$$y_p = d_y (c_y - v)$$

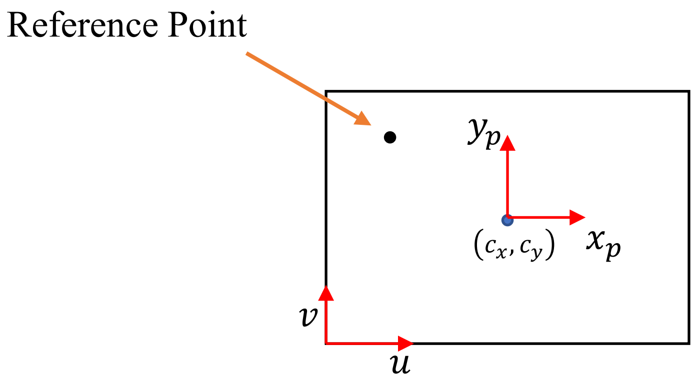
Rectification review – Assignment 2 Phase 1

Use Coordinate System Convention C_2

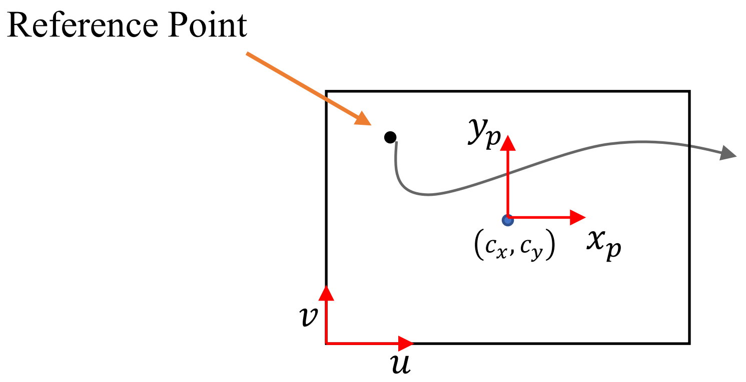
$$\begin{pmatrix} s\tilde{u} \\ s\tilde{v} \\ s \end{pmatrix}_{3\times1} = \overbrace{\underbrace{\begin{pmatrix} \frac{s_x}{d_x} & 0 & c_x \\ 0 & -\frac{1}{d_y} & c_y \\ 0 & 0 & 1 \end{pmatrix}_{3\times3} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix}_{3\times4}}_{\textbf{Camera Matrix } (K)} \underbrace{\begin{pmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix}_{4\times4}}_{\textbf{Extrinsic parameters/matrix } (M)}}^{\textbf{Projection Matrix}} \begin{pmatrix} X \\ Y \\ Z \\ 1 \end{pmatrix}_{4\times1}$$

Backward Projection

Back projection from 2D pixels to 3D coordinates



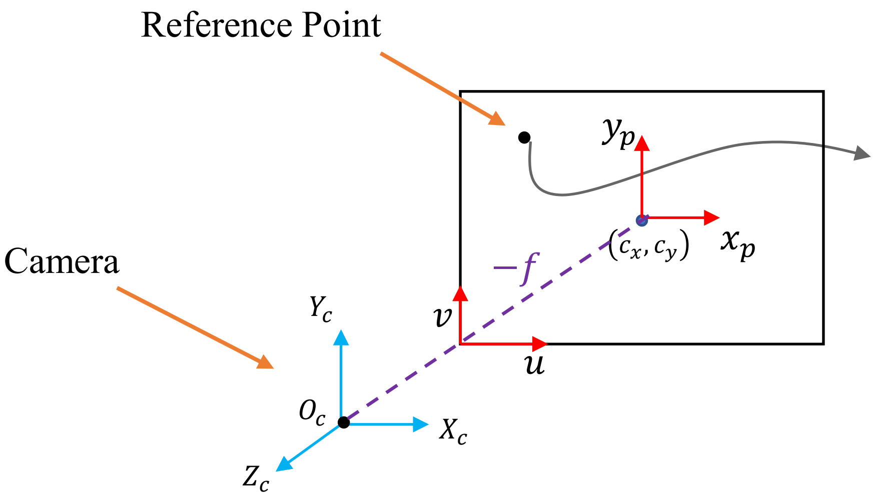
Back projection from 2D pixels to 3D coordinates



$(u, v)^T$ — Image point coordinate

$(x_p, y_p)^T$ — Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)

Back projection from 2D pixels to 3D coordinates

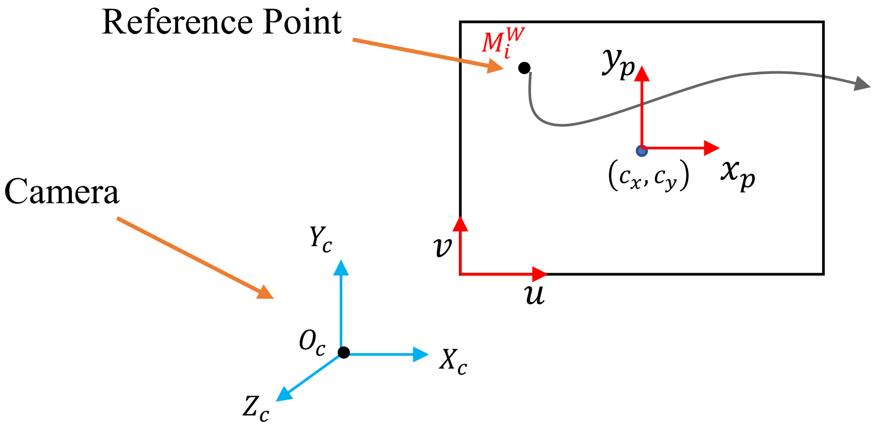


$(u, v)^T$ — Image point coordinate

$(x_p, y_p)^T$ — Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)

$(x_p, y_p, -f)^T$ — Step 2: change from 2D to 3D camera reference coordinate

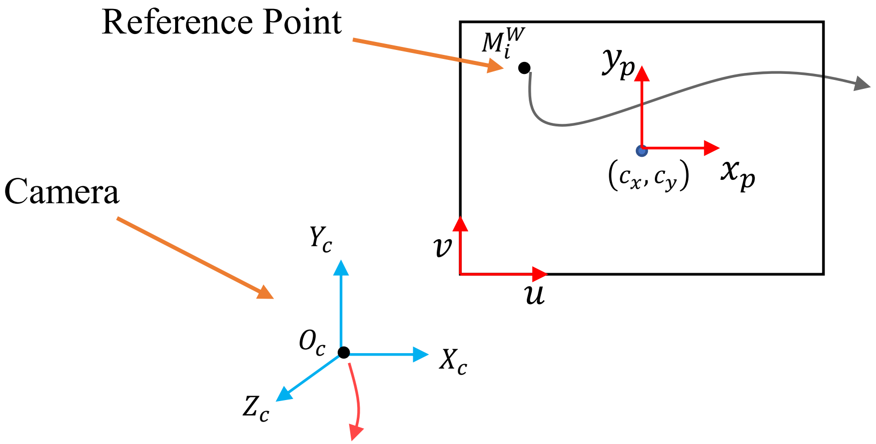
Back projection from 2D pixels to 3D coordinates



$(u, v)^T$	Image point coordinate
$(x_p, y_p)^T$	Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)
$(x_p, y_p, -f)^T$	Step 2: change from 2D to 3D camera reference coordinate
$(R\|t)^{-1}(x_p, y_p, -f)^T$	Step 3: change from 3D camera reference coordinate to 3D world (cube) reference coordinate

$$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T$$

Back projection from 2D pixels to 3D coordinates



$(u, v)^T$	Image point coordinate
$(x_p, y_p)^T$	Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)
$(x_p, y_p, -f)^T$	Step 2: change from 2D to 3D camera reference coordinate
$(R\|t)^{-1}(x_p, y_p, -f)^T$	Step 3: change from 3D camera reference coordinate to 3D world (cube) reference coordinate

$$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T$$

Step 4: change optical center O_c in 3D camera reference coordinate to 3D world reference coordinate.
Compute $O_c^W = (R|t)^{-1}O_c$, where $O_c = [0, 0, 0, 1]^T$

Back projection from 2D pixels to 3D coordinates

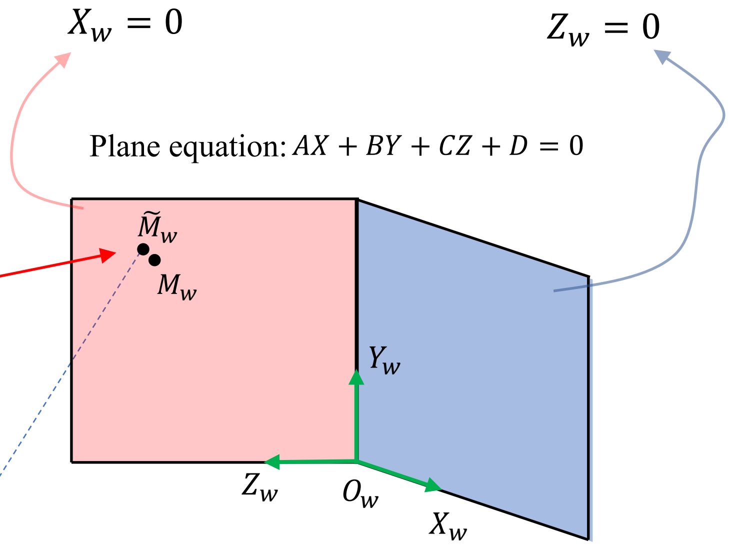
The parametric equation (in WRF) of the ray:

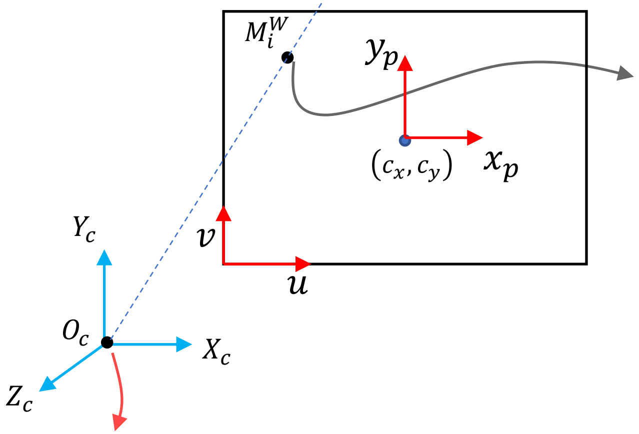
$[O_c^W M_i^W] \rightarrow \boldsymbol{P}(t) = \boldsymbol{O_c^W} + t(\boldsymbol{M_i^W} - \boldsymbol{O_c^W})$

We want:

$\boldsymbol{P}(t) = \boldsymbol{O_c^W} + t(\boldsymbol{M_i^W} - \boldsymbol{O_i^W}) = \widetilde{\boldsymbol{M}}_{\boldsymbol{w}}$

Find t





$(u, v)^T$	Image point coordinate
$(x_p, y_p)^T$	Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)
$(x_p, y_p, -f)^T$	Step 2: change from 2D to 3D camera reference coordinate
$(R\|t)^{-1}(x_p, y_p, -f)^T$	Step 3: change from 3D camera reference coordinate to 3D world (cube) reference coordinate

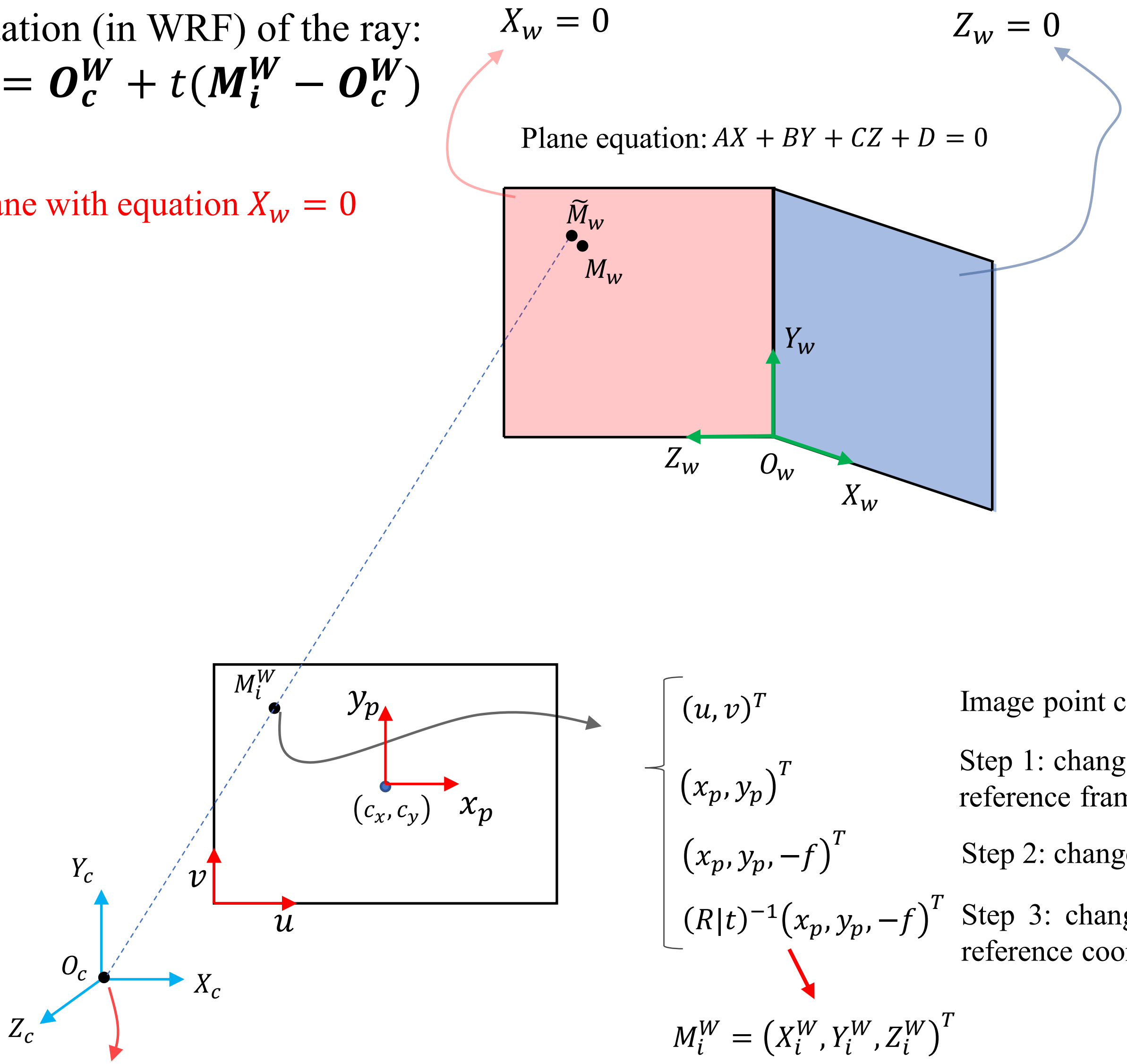
$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T$

Step 4: change optical center O_c in 3D camera reference coordinate to 3D world reference coordinate.
Compute $O_c^W = (R|t)^{-1} O_c$, where $O_c = [0, 0, 0, 1]^T$

Back projection from 2D pixels to 3D coordinates

The parametric equation (in WRF) of the ray:
$[O_c^W M_i^W] \rightarrow \boldsymbol{P}(t) = \boldsymbol{O_c^W} + t(\boldsymbol{M_i^W} - \boldsymbol{O_c^W})$

Step 5: for the left plane with equation $X_w = 0$



$(u, v)^T$	Image point coordinate
$(x_p, y_p)^T$	Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)
$(x_p, y_p, -f)^T$	Step 2: change from 2D to 3D camera reference coordinate
$(R\|t)^{-1}(x_p, y_p, -f)^T$	Step 3: change from 3D camera reference coordinate to 3D world (cube) reference coordinate

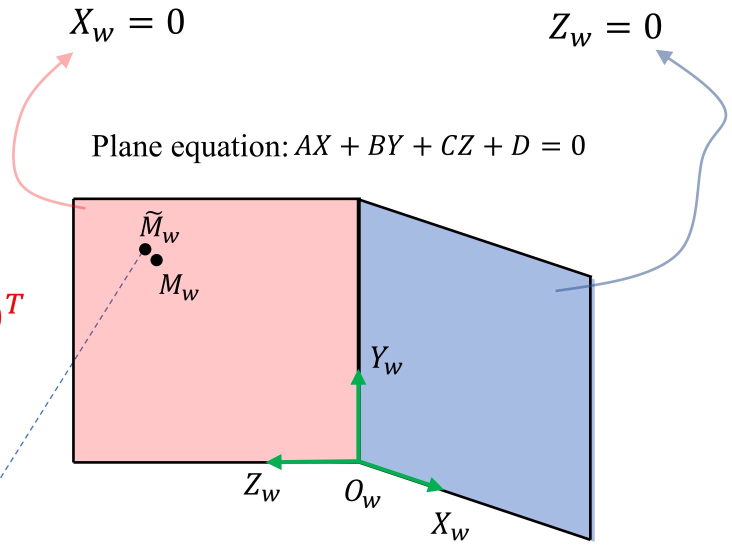
$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T$

Step 4: change optical center O_c in 3D camera reference coordinate to 3D world reference coordinate.
Compute $O_c^W = (R|t)^{-1} O_c$, where $O_c = [0, 0, 0, 1]^T$

Back projection from 2D pixels to 3D coordinates

The parametric equation (in WRF) of the ray:

$[O_c^W M_i^W] \rightarrow \boldsymbol{P}(t) = \boldsymbol{O_c^W} + t(\boldsymbol{M_i^W} - \boldsymbol{O_c^W})$

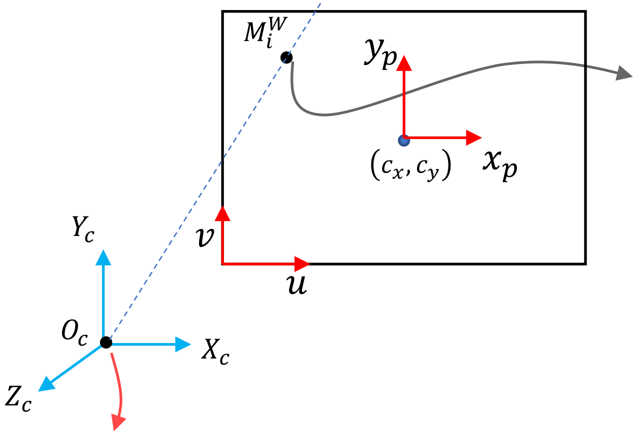


Step 5: for the left plane with equation $X_w = 0$

$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T,\ O_c^W = (X_c^W, Y_c^W, Z_c^W)^T$

$X_c^W + t(X_i^W - X_c^W) = \tilde{X}_w = 0$

Step 6: $t = \frac{-X_c^W}{X_i^W - X_c^W}$



$(u, v)^T$	Image point coordinate
$(x_p, y_p)^T$	Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)
$(x_p, y_p, -f)^T$	Step 2: change from 2D to 3D camera reference coordinate
$(R\|t)^{-1}(x_p, y_p, -f)^T$	Step 3: change from 3D camera reference coordinate to 3D world (cube) reference coordinate

$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T$

Step 4: change optical center O_c in 3D camera reference coordinate to 3D world reference coordinate.
Compute $O_c^W = (R|t)^{-1} O_c$, where $O_c = [0, 0, 0, 1]^T$

Back projection from 2D pixels to 3D coordinates

UNIVERSITY OF AUCKLAND
Waipapa Taumata Rau
NEW ZEALAND

The parametric equation (in WRF) of the ray:

$[O_c^W M_i^W] \rightarrow \boldsymbol{P}(t) = \boldsymbol{O_c^W} + t(\boldsymbol{M_i^W} - \boldsymbol{O_c^W})$

Step 5: for the left plane with equation $X_w = 0$

$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T,\ O_c^W = (X_c^W, Y_c^W, Z_c^W)^T$

$X_c^W + t(X_i^W - X_c^W) = \tilde{X}_w = 0$

Step 6: $t = \frac{-X_c^W}{X_i^W - X_c^W}$

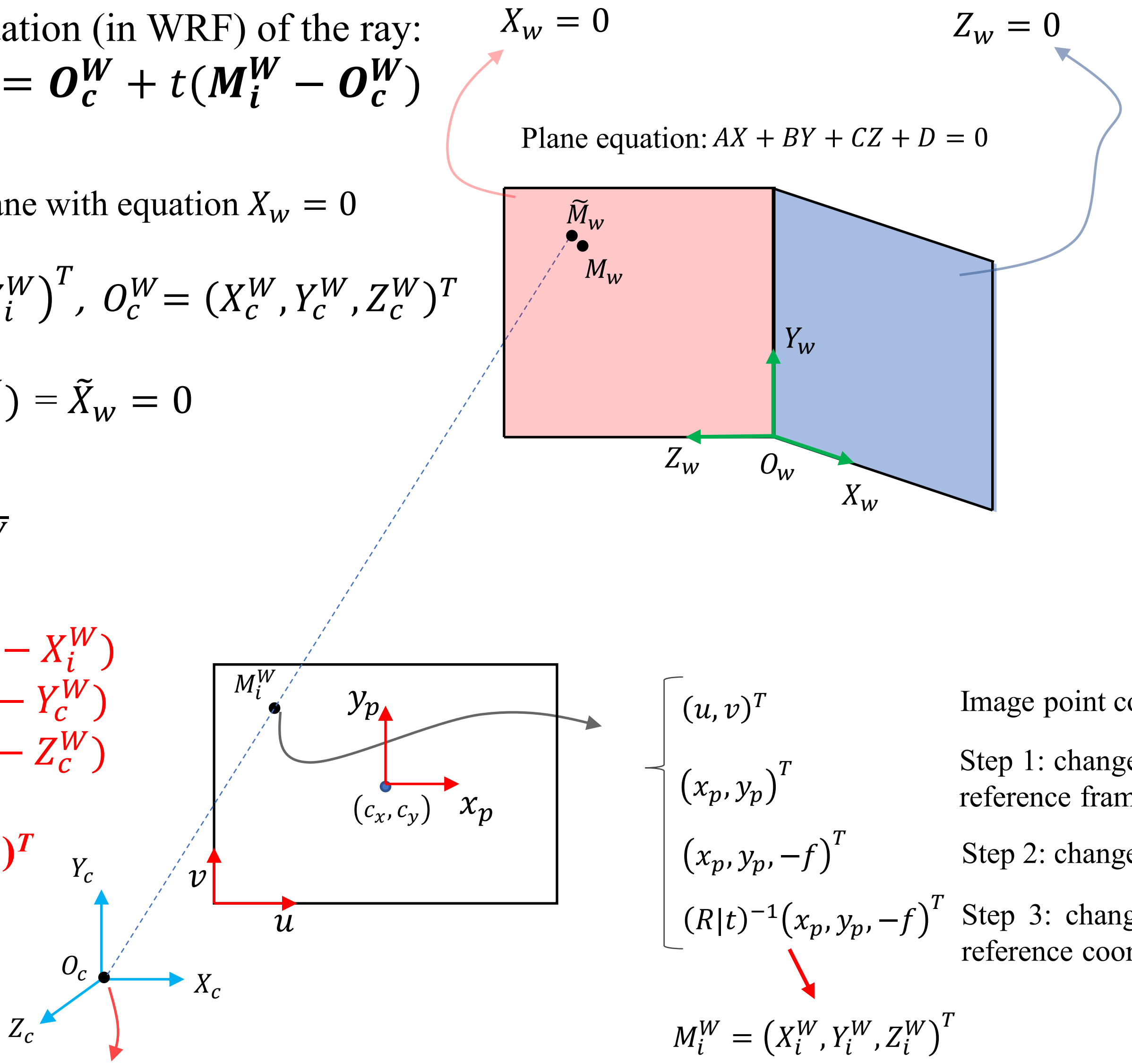
Step 7: compute

$\tilde{X}_w = X_c^W + t(X_i^W - X_i^W)$

$\tilde{Y}_w = Y_c^W + t(Y_i^W - Y_c^W)$

$\tilde{Z}_w = Z_c^W + t(Z_i^W - Z_c^W)$

$\boldsymbol{\tilde{M}_w = (\tilde{X}_w, \tilde{Y}_w, \tilde{Z}_w)^T}$



$(u, v)^T$ Image point coordinate

$(x_p, y_p)^T$ Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)

$(x_p, y_p, -f)^T$ Step 2: change from 2D to 3D camera reference coordinate

$(R|t)^{-1}(x_p, y_p, -f)^T$ Step 3: change from 3D camera reference coordinate to 3D world (cube) reference coordinate

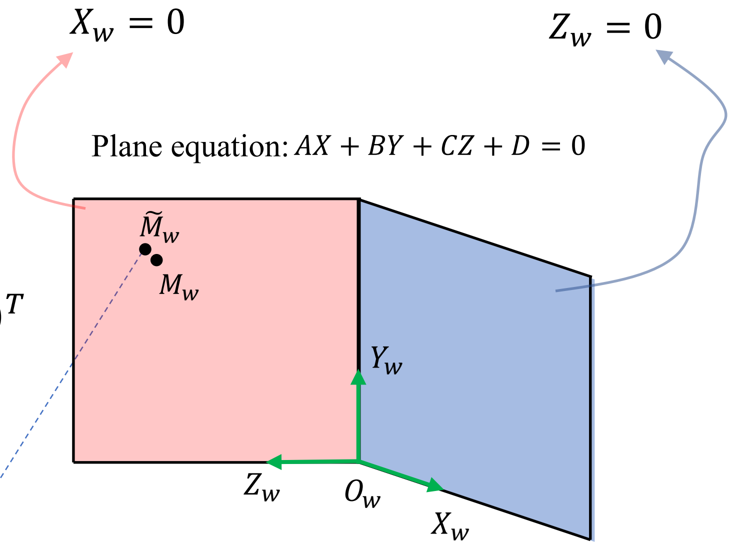
$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T$

Step 4: change optical center O_c in 3D camera reference coordinate to 3D world reference coordinate.
Compute $O_c^W = (R|t)^{-1} O_c$, where $O_c = [0, 0, 0, 1]^T$

Back projection from 2D pixels to 3D coordinates

The parametric equation (in WRF) of the ray:

$[O_c^W M_i^W] \rightarrow \boldsymbol{P}(t) = \boldsymbol{O_c^W} + t(\boldsymbol{M_i^W} - \boldsymbol{O_c^W})$



Step 5: for the left plane with equation $X_w = 0$

$$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T,\ O_c^W = (X_c^W, Y_c^W, Z_c^W)^T$$

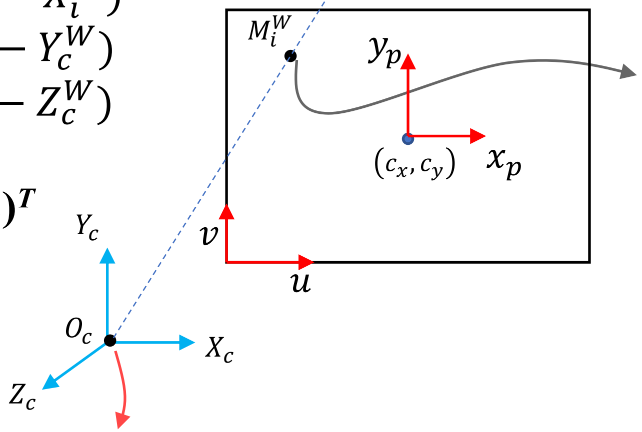
$$X_c^W + t(X_i^W - X_c^W) = \tilde{X}_w = 0$$

Step 6: $t = \frac{-X_c^W}{X_i^W - X_c^W}$

Step 7: compute

$$\tilde{X}_w = X_c^W + t(X_i^W - X_i^W)$$
$$\tilde{Y}_w = Y_c^W + t(Y_i^W - Y_c^W)$$
$$\tilde{Z}_w = Z_c^W + t(Z_i^W - Z_c^W)$$

$$\boldsymbol{\tilde{M}_w = (\tilde{X}_w, \tilde{Y}_w, \tilde{Z}_w)^T}$$



$(u, v)^T$	Image point coordinate
$(x_p, y_p)^T$	Step 1: change to world unit coordinate in *mm* (based on the image reference frame with the origin at the center of the image)
$(x_p, y_p, -f)^T$	Step 2: change from 2D to 3D camera reference coordinate
$(R\|t)^{-1}(x_p, y_p, -f)^T$	Step 3: change from 3D camera reference coordinate to 3D world (cube) reference coordinate

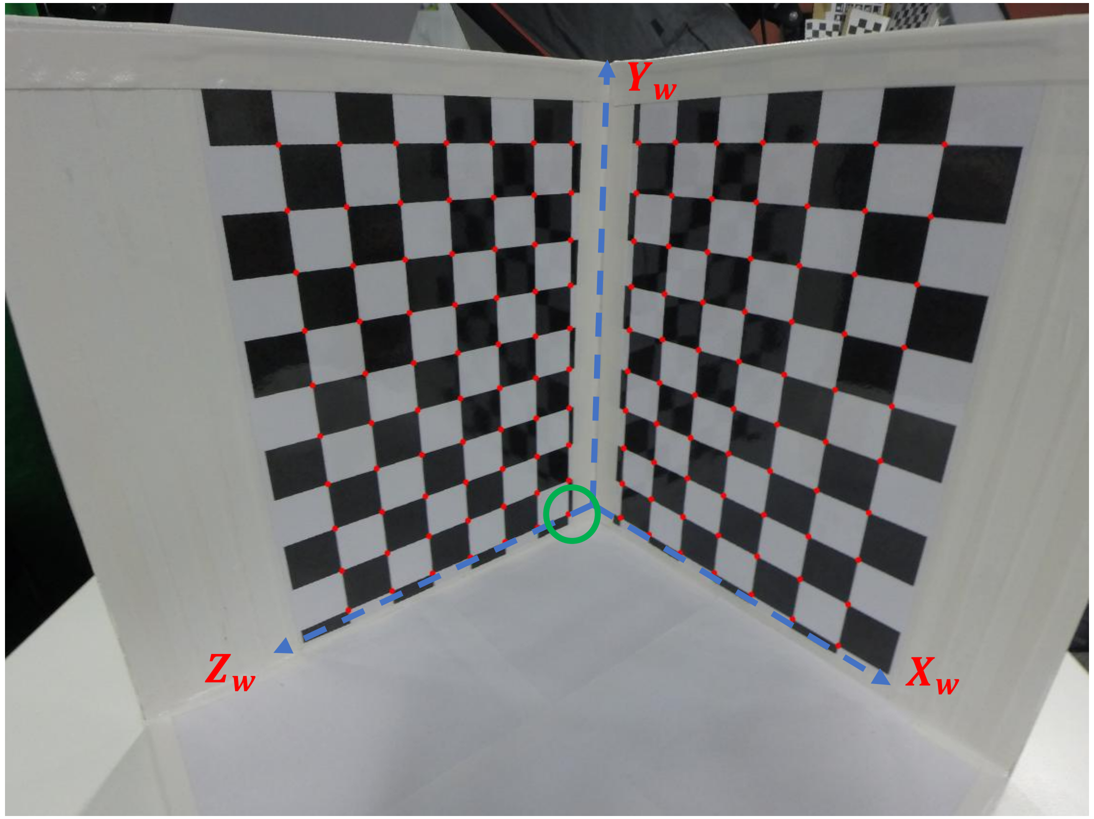
$M_i^W = (X_i^W, Y_i^W, Z_i^W)^T$

Step 4: change optical center O_c in 3D camera reference coordinate to 3D world reference coordinate.
Compute $O_c^W = (R|t)^{-1} O_c$, where $O_c = [0, 0, 0, 1]^T$

Back-projection example

- For the assignment:
- $X_w = 0$ is used for all corners in the left chessboard
- $Z_w = 0$ is used for all corners in the right chessboard
- For instance, consider the first point on the left plane $X_w = 0$.
- $(u, v)^T = (1487.91687011719, 1439.35205078125)^T$
- The corresponding 3D coordinate in the real world (on the left plane, $X_w = 0$) is
  - $(X_w, Y_w, Z_w)^T = (0, 0, 3.4)^T$
- Given $(u, v)^T$, compute the back-projected point on the cube.

$X_w = 0$ $Z_w = 0$



NOTE: we expect to find a coordinate close to $(0, 0, 3.4)^T$

Back-projection example

- Compute $(R|t)^{-1}$
  - $(R|t)^{-1} = \begin{pmatrix} R^T & -R^T t \\ 0^T & 1 \end{pmatrix}$

$$Rt^{-1} = \begin{bmatrix} 0.69358669 & 0.13580093 & -0.70710192 & 38.74681978 \\ 0.05336124 & -0.98883485 & -0.14011019 & 17.95674733 \\ -0.71839410 & 0.06134937 & -0.69309163 & 41.51768039 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

We have already computed calibration parameters:

$$R_t = \begin{bmatrix} 0.69358669 & 0.05336124 & -0.71839410 & 2.01994584 \\ 0.13580093 & -0.98883485 & 0.06134937 & 9.95267017 \\ -0.70710192 & -0.14011019 & -0.69309163 & 58.68943072 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$f = -2.70868944879306$$

$$s_x = 1.001849117285065$$

$$d_x = d_y = 0.00155$$

$$c_x = 1500$$

$$c_y = 1125$$

Back-projection example

- Step 1: Transform the 2D coordinate in pixel into world unit coordinates
  - $(u, v)^T = (1487.91687011719, 1439.35205078125)^T$
  - $x_p = s_x d_x (u - c_x)$
  - $y_p = d_y (v - c_y)$
  - $(x_p, y_p)^T = (-0.019\ldots,\ 0.487\ldots)^T$
- Step 2: Transform $(x_p, y_p)^T$ into CRF
  - $(x_p, y_p, -f)^T = (0.00923\ldots, -0.44892\ldots, -(-2.709\ldots))^T$
- Step 3: Transform $(x_p, y_p, -f)^T$ in CRF into WRF
  - $M_i^W = (R|t)^{-1}(x_p, y_p, -f, 1)^T$
  - $M_i^W = (-0.01876348\ldots, 0.48724568\ldots, 2.70868945\ldots, 1.0)^T$

We have already computed calibration parameters:

$$R_t = \begin{bmatrix} 0.69358669 & 0.05336124 & -0.71839410 & 2.01994584 \\ 0.13580093 & -0.98883485 & 0.06134937 & 9.95267017 \\ -0.70710192 & -0.14011019 & -0.69309163 & 58.68943072 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$f = -2.70868944879306$$

$$s_x = 1.001849117285065$$

$$d_x = d_y = 0.00155$$

$$c_x = 1500$$

$$c_y = 1125$$

Back-projection example

- Step 4: Transform the optical centre coordinates in CRF into the corresponding one in WRF.
  $O_c^W = (R|t)^{-1}\,(0,0,0,1)^T$
  $O_c^W = (36.885\ldots, 17.094\ldots, 39.684\ldots, 1.0)^T$

- Steps 5-6: Create the parametric equation (in WRF) of the ray $[O_w^c M_w^I]$
  - $O_c^W = (X_c^W, Y_c^W, Z_c^W), M_i^W = \left(X_i^W, Y_i^W, Z_i^W\right)$
  - $O_c^W + t\left(M_i^W - O_c^W\right) \quad \begin{cases} X_c^W + t\left(X_i^W - X_c^W\right) \\ Y_c^W + t\left(Y_i^W - Y_c^W\right) \\ Z_c^W + t\left(Z_i^W - Z_c^W\right) \end{cases}$
  - Since we know that $(u, v)^T$ will be back-projected onto left plane $X_w = 0$, we need to use first equation
  - $X_c^W + t\left(X_i^W - X_c^W\right) \rightarrow \; t = \frac{-X_c^W}{X_i^W - X_c^W} = 20.807\ldots$

We have already computed calibration parameters:

$$R_t = \begin{bmatrix} 0.69358669 & 0.05336124 & -0.71839410 & 2.01994584 \\ 0.13580093 & -0.98883485 & 0.06134937 & 9.95267017 \\ -0.70710192 & -0.14011019 & -0.69309163 & 58.68943072 \\ 0 & 0 & 0 & 1 \end{bmatrix}$$

$$f = -2.70868944879306$$

$$s_x = 1.001849117285065$$

$$d_x = d_y = 0.00155$$

$$c_x = 1500$$

$$c_y = 1125$$

Back-projection example

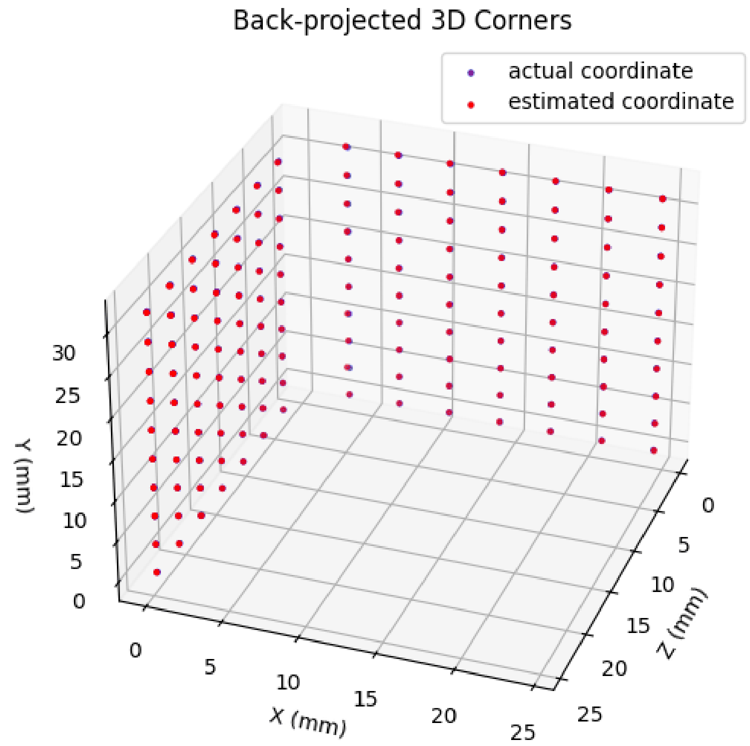
- Step 7: Once t is computed, we can compute the backprojected 3D coordinate on the plane as follows

$$\tilde{X}_w = X_c^W + t(X_i^W - X_i^W)$$
$$\tilde{Y}_w = Y_c^W + t(Y_i^W - Y_c^W)$$
$$\tilde{Z}_w = Z_c^W + t(Z_i^W - Z_c^W)$$

back-projected $\left(\tilde{X}_w, \tilde{Y}_w, \tilde{Z}_w\right)^T = (0.0, 0.014\ldots, 3.357\ldots)^T$

original $(X_w, Y_w, Z_w)^T = (0.0, 0.0, 3.4)^T$

- After back-projecting all the corners in the left chessboard, you will back-project all the corners in the right chessboard using $Z_w = 0$



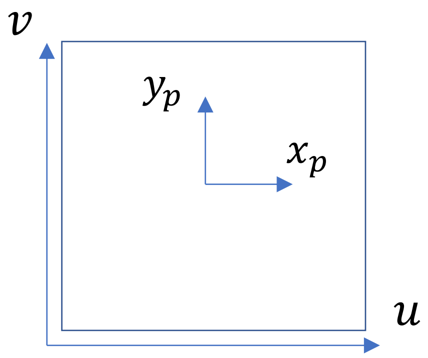
Backward Projection Quiz Question

After calibration of camera $\boldsymbol{C}$, it was determined that the focal length $\boldsymbol{f}$ was $\boldsymbol{-5\ mm}$, the optical center $c = [119,\ 194]^{\mathrm{T}}$ in pixels and there was no distortion. The pixels were determined to be square with side lengths of $\boldsymbol{0.001\ mm}$ on camera's image sensor. The extrinsic parameters were as follows:

$$P = \begin{bmatrix} 0.438 & 0.000 & 0.899 & -9.000 \\ 0.000 & 1.000 & 0.000 & 2.000 \\ -0.899 & 0.000 & 0.438 & 22.000 \\ 0.000 & 0.000 & 0.000 & 1.000 \end{bmatrix}$$

Assume that Tsai calibration was performed, and that corner features of a calibration grid were used for this calibration. If one of these features was projected from the calibration plane $Z_w = 0$ to be at pixel location $p = [174,\ 73]^{\mathrm{T}}$ within the image, what is the expected 3D location of this corner within the scene (assuming no calibration error, Z_w is pointing away from the image, $s_x = 1$)?

C_1 Coordinate System Convention:



$$x_p = s_x d_x(u - c_x) = 0.055$$
$$y_p = d_y(v - c_y) = -0.121$$

$$f = -5\ mm;\ c_x = 119, c_y = 194;\ u = 174, v = 73;\ d_x = 0.001, d_y = 0.001;\ s_x = 1$$

$$O_c^W = P^{-1}\,(0,0,0,1)^T$$
$$O_c^W = (23.72, -2.0\,, -1.545)$$

$$M_i^W = P^{-1}\left(x_p, y_p, -f, 1\right)^T$$
$$M_i^W = P^{-1}(0.055, -0.121, -(-5), 1)^T$$
$$M_i^W = (19.249, -2.121, 0.694)$$

Since $Z_w = 0$ $\quad t = \dfrac{-Z_c^W}{Z_i^W - Z_c^W} = \dfrac{-(-1.545)}{0.694 - (-1.545)} = 0.69$

$$\tilde{X}_w = X_c^W + t(X_i^W - X_i^W)$$
$$\tilde{X}_w = 23.72 + 0.69(19.249 - 23.72)$$
$$\tilde{X}_w = 20.6$$

$$\tilde{Y}_w = Y_c^W + t(Y_i^W - Y_c^W)$$
$$\tilde{Y}_w = -2.0 + 0.69(-2.121 - (-2.0))$$
$$\tilde{Y}_w = -2.1$$

$$\tilde{Z}_w = Z_c^W + t(Z_i^W - Z_c^W)$$
$$\tilde{Z}_w = 0.0$$

Do not round your intermediate results in your quiz/assignment! (Here is for the saving spaces)

Tsai Calibration Errors

Calibration error in the 2D image

- **Main goal**: given all 2D pixels (u_i, v_i) and their corresponding 3D points (X_i, Y_i, Z_i) in the real world (cube), compute calibration error in the image, where $0 \leq i \leq n$, and n is the number of estimated corners.

- Compute projected points from 3D world in *mm* to 2D image in pixel

$$\begin{pmatrix} \tilde{u}_i \\ \tilde{v}_i \\ 1 \end{pmatrix} = \begin{pmatrix} \frac{s_x}{d_x} & 0 & c_x \\ 0 & \frac{1}{d_y} & c_y \\ 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} f & 0 & 0 & 0 \\ 0 & f & 0 & 0 \\ 0 & 0 & 1 & 0 \end{pmatrix} \begin{pmatrix} r_{11} & r_{12} & r_{13} & t_x \\ r_{21} & r_{22} & r_{23} & t_y \\ r_{31} & r_{32} & r_{33} & t_z \\ 0 & 0 & 0 & 1 \end{pmatrix} \begin{pmatrix} X_i \\ Y_i \\ Z_i \\ 1 \end{pmatrix}$$

C_1 Coordinate System Convention

- Compute RMSE in the image (other options: MAE, MSE, SD)

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^{n} (u_i - \tilde{u}_i)^2 + (v_i - \tilde{v}_i)^2}$$

Calibration error in the cube

- **Main goal**: given all 2D pixels (u_i, v_i) and their corresponding 3D points (X_i, Y_i, Z_i) in the real world (cube), compute calibration error in the cube, where $0 \leq i \leq n$, and n is the number of estimated corners.
- Compute back-projected points from (u_i, v_i) in pixels into $(\tilde{X}_i, \tilde{Y}_i, \tilde{Z}_i)$ on the cube
- Compute RMSE in the cube (other options: MAE, MSE, SD, …)

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(X_i - \tilde{X}_i)^2 + (Y_i - \tilde{Y}_i)^2 + (Z_i - \tilde{Z}_i)^2}$$

```
print(tsai.cubeError)
```

```
0.04645539611732293
```

Stereo calibration error

Given the left and right stereo images taken by the left and right cameras, respectively, compute the stereo calibration error.

1. Detect all the corners in both left and right images.
2. Compute Tsai calibration parameters for both left and right cameras.
3. For all matching corners in the left and right images, do the following:
    1. Define the parametric equation of the ray $r_{left} = [O_{cl}^{Wl} M_i^{Wl}]$ for the left camera and image
    2. Define the parametric equation of the ray $r_{right} = [O_{cr}^{Wr} M_i^{Wr}]$ for the right camera and image
    3. Find the intersection of the rays r_{left} and r_{right}
    4. If t_1 and t_2 do not give the same points, take the average of the estimated points
    5. Let the intersection point be $(\tilde{X}_i, \tilde{Y}_i, \tilde{Z}_i)$
4. Compute stereo calibration error as follows, where (X_i, Y_i, Z_i) are the actual 3D coordinates in the real world (cube)

O_c^W: camera optical centre in WRF
M_i^W: image coordinate in WRF

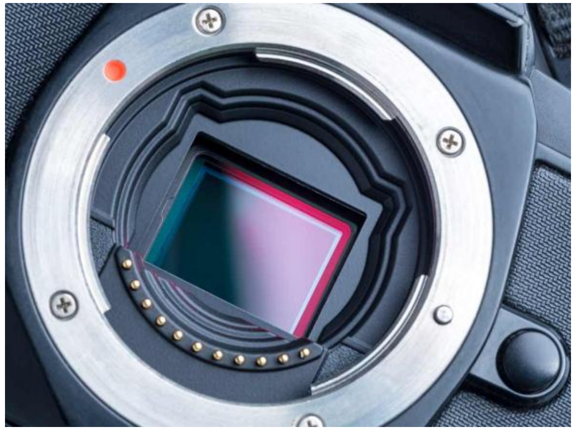
$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(X_i - \tilde{X}_i)^2 + (Y_i - \tilde{Y}_i)^2 + (Z_i - \tilde{Z}_i)^2}$$

Distortion

Distortion

- **How to compute $pixel\_size$ from camera sensor size**?

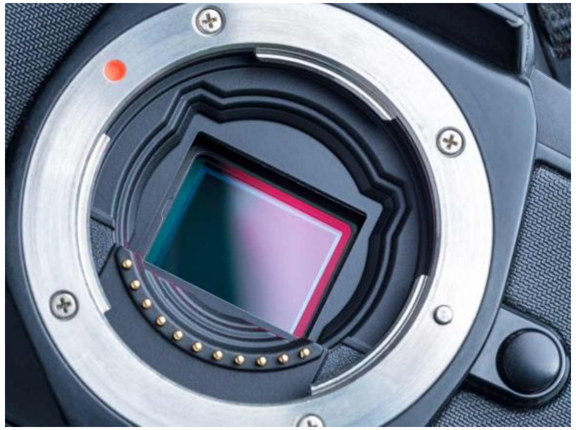
- Camera sensors are made up of millions of tiny light-sensitive pixels, which are responsible for capturing the image. Each pixel measures the intensity of the light that falls on it and generates an electrical signal. The camera's processor then converts these signals into a digital image.



Distortion

- **How to compute $pixel\_size$ from camera sensor size**?
- Camera sensors are made up of millions of tiny light-sensitive pixels, which are responsible for capturing the image. Each pixel measures the intensity of the light that falls on it and generates an electrical signal. The camera's processor then converts these signals into a digital image.

- Given the camera sensor size in millimeter as $cs_x \times cs_y$ and its image resolution as $w \times h$
- $pixel\_size\_x = cs_x/w$
- $pixel\_size\_y = cs_y/h$
- $pixel\_size = (pixel\_size\_x + pixel\_size\_y)/2$



Distortion Removal Questions

Example-1

- *Consider a* (5000×3750) *pixel camera with a sensor size of* $5.0\,mm \times 3.75\,mm$ and Pincushion distortion. *Given the first-order radial distortion parameter* $\kappa_1 = -1.2E - 3\,mm^{-2}$, *compute the undistorted pixel coordinates for the point* M *with the distorted pixel coordinates* $(0{,}0)$. *Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards. (Round to the nearest pixel)*

Example-1

- *Consider a* (5000×3750) *pixel camera with a sensor size of* $5.0\,mm \times 3.75\,mm$ and Pincushion distortion. *Given the first-order radial distortion parameter* $\kappa_1 = -1.2E-3\,mm^{-2}$, *compute the undistorted pixel coordinates for the point* M *with the distorted pixel coordinates* $(0{,}0)$. *Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards. (Round to the nearest pixel)*

  - $w = 5000, h = 3750$
  - $cs_x = 5.0\,mm,\ cs_y = 3.75\,mm$
  - κ_1^m = *-0.0012* mm^{-2}
  - $x_d = 0,\ y_d = 0$
  - ***We need to compute*** $\boldsymbol{x_u}$ ***and*** $\boldsymbol{y_u}$

Example-1

- *Consider a* (5000×3750) *pixel camera with a sensor size of* $5.0\,mm \times 3.75\,mm$ and Pincushion distortion. *Given the first-order radial distortion parameter* $\kappa_1 = -1.2E-3\,mm^{-2}$, *compute the undistorted pixel coordinates for the point M with the distorted pixel coordinates* $(0{,}0)$. *Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards. (Round to the nearest pixel)*

- $w = 5000, h = 3750$
- $cs_x = 5.0\,mm,\ cs_y = 3.75\,mm$
- κ_1^m = *-0.0012* mm^{-2}
- $x_d = 0,\ y_d = 0$
- ***We need to compute*** $\boldsymbol{x_u}$ ***and*** $\boldsymbol{y_u}$

To get x_u and y_u we need κ_1 in pixel unit

We κ_1 in mm unit, so we need the conversion

For the conversion, we need **pixel size**

Example-1

- *Consider a* (5000×3750) *pixel camera with a sensor size of* $5.0\,mm \times 3.75\,mm$ and Pincushion distortion. *Given the first-order radial distortion parameter* $\kappa_1 = -1.2E - 3\,mm^{-2}$, *compute the undistorted pixel coordinates for the point* M *with the distorted pixel coordinates* $(0{,}0)$. *Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards. (Round to the nearest pixel)*

- $w = 5000, h = 3750$
- $cs_x = 5.0\,mm,\ cs_y = 3.75\,mm$
- κ_1^m = -0.0012 mm^{-2}
- $x_d = 0,\ y_d = 0$
- ***We need to compute*** $\boldsymbol{x_u}$ ***and*** $\boldsymbol{y_u}$

$$c_x = \frac{5000}{2} = 2500, c_y = \frac{3750}{2} = 1875$$

$pixel\_size\_x$ = $5/5000$ = 0.001
$pixel\_size\_y$ =$3.75/3750$= 0.001
$pixel\_size$ = (0.001+ 0.001)/2 = 0.001

κ_1^p = $-0.0012 \times (0.001)^2\ px^{-2}$

Example-1

- *Consider a* (5000×3750) *pixel camera with a sensor size of* $5.0\,mm \times 3.75\,mm$ and Pincushion distortion. *Given the first-order radial distortion parameter* $\kappa_1 = -1.2E-3\,mm^{-2}$, *compute the undistorted pixel coordinates for the point M with the distorted pixel coordinates* $(0,0)$. *Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards. (Round to the nearest pixel)*

- $w = 5000, h = 3750$
- $cs_x = 5.0\,mm,\ cs_y = 3.75\,mm$
- $\kappa_1^m = -0.0012\,mm^{-2}$
- $x_d = 0,\ y_d = 0$
- ***We need to compute*** $\boldsymbol{x_u}$ ***and*** $\boldsymbol{y_u}$
- $c_x = 2500,\ c_y = 1875$
- *pixel_size*= 0.001
- $\kappa_1^p = -0.0012 \times (0.001)^2\,px^{-2}$

Example-1

- *Consider a* (5000×3750) *pixel camera with a sensor size of* $5.0\,mm \times 3.75\,mm$ and Pincushion distortion. *Given the first-order radial distortion parameter* $\kappa_1 = -1.2E-3\,mm^{-2}$, *compute the undistorted pixel coordinates for the point* M *with the distorted pixel coordinates* $(0{,}0)$. *Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards. (Round to the nearest pixel)*

- $w = 5000, h = 3750$
- $cs_x = 5.0\,mm,\ cs_y = 3.75\,mm$
- κ_1^m = *-0.0012* mm^{-2}
- $x_d = 0,\ y_d = 0$
- ***We need to compute*** $\boldsymbol{x_u}$ ***and*** $\boldsymbol{y_u}$
- $c_x = 2500,\ c_y = 1875$
- *pixel_size*= 0.001
- $\kappa_1^p = -0.0012 \times (0.001)^2\ px^{-2}$

$$x_u = (x_d - c_x)\left(1 + \kappa_1^p\left((x_d - c_x)^2 + \left(y_d - c_y\right)^2\right)\right) + c_x$$

$$y_u = (y_d - c_y)\left(1 + \kappa_1^p\left((x_d - c_x)^2 + \left(y_d - c_y\right)^2\right)\right) + c_y$$

$x_u = 29.296875 = 29 \qquad y_u = 21.97265625 = 22$

Example-2

- Consider an image with the center pixel coordinate located at point $(1500,1125)$ and Barrel distortion. Given that the square pixel is $(0.002\ mm)$ wide, the distorted pixel is located at $(2351,1155)$, and we want the undistorted pixel (x_u, y_u) to differ only by one pixel. Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards.

What is the value of κ_1^m given the above conditions.

Example-2

- Consider an image with the center pixel coordinate located at point $(1500,1125)$ and Barrel distortion. Given that the square pixel is $(0.002\ mm)$ wide, the distorted pixel is located at $(2351,1155)$, and we want the undistorted pixel (x_u, y_u) to differ only by one pixel. Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards.

What is the value of κ_1^m given the above conditions.

- c_x=1500, $c_y = 1125$
- $pixel\_size$= 0.002 mm
- $x_d = 2351$, $y_d = 1155$
- $\Delta = 1\ pixel$
- κ_1^m is to be computed

$$\boxed{\kappa_1 = \frac{\Delta}{r_d^3}}$$

Example-2

- Consider an image with the center pixel coordinate located at point $(1500,1125)$ and Barrel distortion. Given that the square pixel is $(0.002\ mm)$ wide, the distorted pixel is located at $(2351,1155)$, and we want the undistorted pixel (x_u, y_u) to differ only by one pixel. Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards.

What is the value of κ_1^m given the above conditions.

- c_x=1500, $c_y = 1125$
- $pixel\_size$= 0.002 mm
- $x_d = 2351$, $y_d = 1155$
- $\Delta = 1\ pixel$
- κ_1^m is to be computed

$$\boxed{\kappa_1 = \frac{\Delta}{r_d^3}}$$

Solution-1

$$r_d = \sqrt{(x_d - c_x)^2 + (y_d - c_y)^2} = \sqrt{(2351 - 1500)^2 + (1155 - 1125)^2}$$

$$r_d = 851.528\ in\ pixel\ unit$$

$$\kappa_1^p = \frac{\Delta}{r_d^3} = \frac{1}{(851.528)^3} = 1.619e - 09\ px^{-2}$$

$$\kappa_1^m = \frac{\kappa_1^p}{(pixel\_size)^2} = \frac{1.619e - 09}{(0.002)^2} = 4.04e - 4\ mm^{-2}$$

Example-2

- Consider an image with the center pixel coordinate located at point (1500,1125) and Barrel distortion. Given that the square pixel is ($0.002\ mm$) wide, the distorted pixel is located at (2351,1155), and we want the undistorted pixel (x_u, y_u) to differ only by one pixel. Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards.

What is the value of κ_1^m given the above conditions.

- c_x=1500, $c_y = 1125$
- $pixel\_size$= $0.002\ mm$
- $x_d = 2351$, $y_d = 1155$
- $\Delta = 1\ pixel$
- κ_1^m is to be computed

$$\boxed{\kappa_1 = \frac{\Delta}{r_d^3}}$$

Solution-2

$$r_d = \sqrt{(x_d - c_x)^2 + \left(y_d - c_y\right)^2} = \sqrt{(2351 - 1500)^2 + (1155 - 1125)^2}$$

$$r_d = 851.528\ in\ pixel\ unit$$

$$r_d = 851.528 \times 0.002 = 1.703\ mm$$

$$\Delta = 1 \times 0.002 = 0.002\ mm$$

$$\kappa_1^m = \frac{\Delta}{r_d^3} = \frac{0.002}{(1.703)^3} = 4.04e - 4\ mm^{-2}$$

Example-3

- Consider a 4000×3000 pixel camera with a sensor size of $8.0\ mm \times 6.0\ mm$ and Barrel distortion. Compute the maximum first-order radial distortion parameter κ1:max in mm^{-2} such that the maximal distance between the distorted and undistorted coordinates of any point in the image is kept below the pixel width. Assume the image origin is at the bottom-left, x is pointing rightwards, y is pointing upwards.

Example-3

- Consider a 4000×3000 pixel camera with a sensor size of $8.0\ mm \ \times 6.0\ mm$ and Barrel distortion. Compute the maximum first-order radial distortion parameter κ1:max in mm^{-2} such that the maximal distance between the distorted and undistorted coordinates of any point in the image is kept below the pixel width. Assume the image origin is at the bottom-left, *x* is pointing rightwards, *y* is pointing upwards.
- $w = 4000, h = 3000$
- $cs_x = 8.0\ mm,\ \ cs_y = 6\ mm$
- $x_d = 4000,\ y_d = 3000$
- $\Delta = 1\ pixel$
- κ1:max in in mm^{-2} → $UNKNOWN$

$$\boxed{\kappa_1 = \frac{\Delta}{r_d^3}}$$

Example-3

- Consider a 4000×3000 pixel camera with a sensor size of $8.0\ mm \times 6.0\ mm$ and Barrel distortion. Compute the maximum first-order radial distortion parameter κ1:max in mm^{-2} such that the maximal distance between the distorted and undistorted coordinates of any point in the image is kept below the pixel width. Assume the image origin is at the bottom-left, *x* is pointing rightwards, *y* is pointing upwards.

- $w = 4000, h = 3000$
- $cs_x = 8.0\ mm,\ \ cs_y = 6\ mm$
- $x_d = 4000,\ y_d = 3000$
- $\Delta = 1\ pixel$
- κ1:max in in mm^{-2} → *UNKNOWN*

$$\boxed{\kappa_1 = \frac{\Delta}{r_d^3}}$$

$$r_d = \sqrt{(x_d - c_x)^2 + \left(y_d - c_y\right)^2} = \sqrt{(4000 - 2000)^2 + (3000 - 1500)^2}$$

$$r_d = 2500\ pixels$$

$$\kappa_1^p = \frac{\Delta}{r_d^3} = \frac{1}{(2500)^3} = 6.4E - 11\ px^{-2}$$

$$\kappa_1^m = \frac{\kappa_1^p}{(pixel\_size)^2}$$

$pixel\_size\_x = 8/4000 = 0.002$
$pixel\_size\_y = 6/3000 = 0.002$
$pixel\_size = 0.002$

$$\kappa_1^m = 1.6 \times 10^{-5}\ mm^{-2}$$